

Mapping EER to Relational Model

Relational Model

- Relational model uses collection of **tables** to represent both **data** and the **relationships** among those data.
- a table is a two-dimensional structure made up of *rows (tuples, records)* and *columns (attributes, fields)*

StudentID	Activity	Fee
100	Skiing	200
150	Swimming	50
175	Squash	50
200	Swimming	50

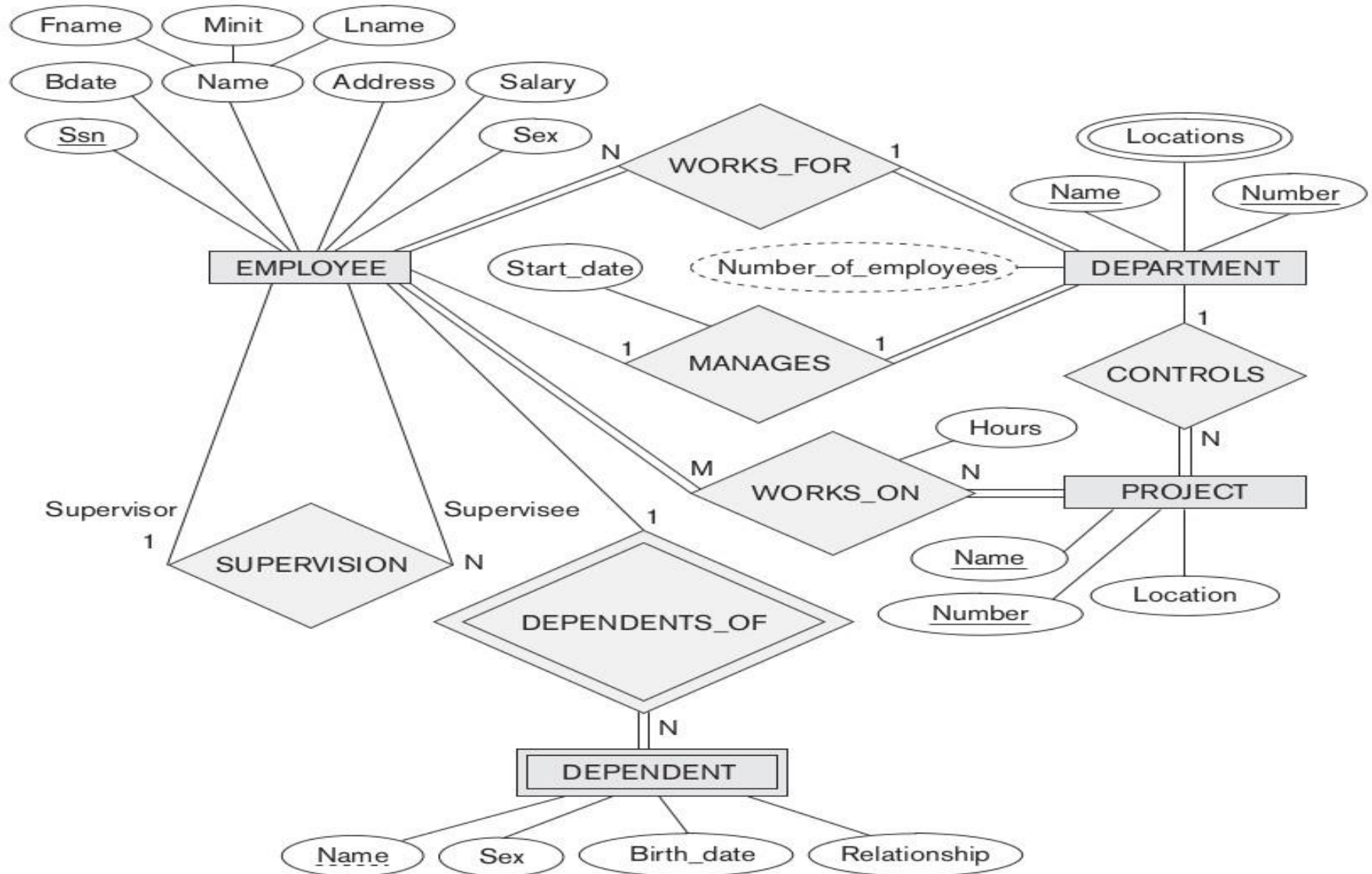
Table Characteristics

- each row is unique and stores data about one entity
- each column has a unique attribute name
- each column (attribute) description (*metadata*) is stored in the database
- Access metadata is stored and manipulated via the Table Design View grid
- all entries in a column have the same data type
- Access examples: Text(50), Number(Integer), Date/Time
- each cell contains atomic data: no lists or sub-tables

Mapping ER to Relational Example

Figure 9.1

The ER conceptual schema diagram for the COMPANY database.



Mapping ER to Relational Example

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	<u>Plocation</u>	Dnum
-------	----------------	------------------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Figure 9.2

Result of mapping the COMPANY ER schema into a relational database schema.

DIFFERENT KEYS

1. **Super Key:** A super key is a set of attributes that can identify each tuple uniquely in the given relation. A super key is not restricted to have any specific number of attributes. Thus, a super key may consist of any number of attributes.
2. **Candidate Key:** If there is more than one possible choice of primary key; then each possible choice is called a candidate key. A set of minimal attribute(s) that can identify each tuple uniquely in the given relation is called as a candidate key. A minimal super key is called as a candidate key. Candidate keys that are left unimplemented or unused after implementing the primary key are called as **alternate keys**.
3. **Primary Keys:** a primary key is an attribute or a collection of attributes whose value(s) uniquely identify each row in a relation. A primary key is a candidate key that the database designer selects while designing the database. **a primary key should be minimal: it should not contain unnecessary attributes**
4. **composite key or concatenated key:** If primary key is a combination of values then such a multi-attribute primary key is called a composite key or concatenated key
5. **Partial Key:** A partial key is one or more fields that form a part of a primary key. Thus a partial key does not uniquely identify a row, but can be combined with another partial key.
6. **Foreign Keys:** A foreign key is an attribute or a collection of attributes whose value are pointing to primary key/ candidate key of some related record. They act as a cross-reference between tables.
7. **Unique Key:** Unique key is a key with the following properties- It is unique for all the records of the table. Once assigned, its value can not be changed i.e. it is non-updatable. It may have a NULL value. The best example of unique key is [Adhaar Card Numbers](#). An updatable unique key is known as **surrogate key**.

Algorithm/ Steps

- **ER-to-Relational Mapping Algorithm**
 - Step 1: Mapping of Regular Entity Types
 - Step 2: Mapping of Weak Entity Types
 - Step 3: Mapping of Binary 1:1 Relation Types
 - Step 4: Mapping of Binary 1:N Relationship Types.
 - Step 5: Mapping of Binary M:N Relationship Types.
 - Step 6: Mapping of Multivalued attributes.
 - Step 7: Mapping of N-ary Relationship Types.
- **Mapping EER Model Constructs to Relations**
 - Step 8: Options for Mapping Specialization or Generalization.
 - Step 9: Mapping of Union Types (Categories).

ER-to-Relational Mapping Algorithm

- **Step 1: Mapping of Regular Entity Types.**
 - For each **regular (strong) entity type E** in the ER schema, create a **relation R** that includes all the **simple attributes of E**.
 - Choose one of the key attributes of E as the **primary key for R**.
 - If the chosen key of **E is composite**, the set of simple attributes that **form it will together form the primary key of R**.
- **Example:** We create the relations **EMPLOYEE**, **DEPARTMENT**, and **PROJECT** in the relational schema corresponding to the regular entities in the ER diagram.
 - SSN, DNUMBER, and PNUMBER are the primary keys for the relations EMPLOYEE, DEPARTMENT, and PROJECT as shown.

Employee	<u>SSN</u>	Bdate	FName	Lname	Minit	Address	Salary	Sex
-----------------	------------	-------	-------	-------	-------	---------	--------	-----

Department	Dno	DName
-------------------	-----	-------

Project	Pno	PName	Location
----------------	-----	-------	----------

ER-to-Relational Mapping Algorithm

- **Step 2: Mapping of Weak Entity Types**
 - For each **weak entity type W** in the ER schema with owner entity type E, create a **relation R** & include all **simple attributes** (or simple components of composite attributes) of W as attributes of R.
 - Also, include as **foreign key attributes** of R the **primary key attribute(s)** of the relation(s) that correspond to the owner entity type(s).
 - The primary key of R is the **combination of the primary key(s)** of the owner(s) and the **partial key of the weak entity type W**, if any.
- **Example:** Create the relation **DEPENDENT** in this step to correspond to the weak entity type DEPENDENT.
 - Include the primary key **SSN of the EMPLOYEE** relation as a **foreign key attribute of DEPENDENT** (renamed to **ESSN**).
 - The primary key of the DEPENDENT relation is the combination {**ESSN, DEPENDENT_NAME**} because DEPENDENT_NAME is the partial key of DEPENDENT.

Dependent	<u>ESSN</u>	DepName	Sex	B_date	Relationship
-----------	-------------	---------	-----	--------	--------------

ER-to-Relational Mapping Algorithm

- **Step 3: Mapping of Binary 1:1 Relation Types**

For each **binary 1:1 relationship** type R in the ER schema, identify S and T that correspond to the entity types participating in R. Let the relationship be of the form **[S] ———^{total}<R>^{partial} ——— [T]**.. There are three possible approaches:

1. **Foreign Key approach:** The **primary key of T(partial)** is added as a **foreign key in S(total relationship)** along with attributes of R. If only one of the entities has total participation it's better to call it in the role of S, to avoid null attributes. If neither entity has total participation nulls may be unavoidable.
 - **Example:** 1:1 relation **MANAGES** is mapped by choosing the participating entity type **DEPARTMENT** to serve in the role of S, because its participation in the MANAGES relationship type is total.
2. **Merged relation option:** An alternate mapping of a 1:1 relationship type is possible by merging the two entity types and the relationship into a single relation. This may be appropriate when **both participations are total**.
3. **Cross-reference or relationship relation option:** The third alternative is to set up a third relation R for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types. A separate relation represents R; each tuple is a foreign key from S and a foreign key from T. Any attributes of R are also added to this relation. Here foreign keys should CASCADE.

Department	Dno	Dname	MgrSSN
------------	-----	-------	--------

ER-to-Relational Mapping Algorithm

- **Step 4: Mapping of Binary 1:N Relationship Types.**
 - Let the relationship be of the form **[S]—^N<R>¹—[T]**.
 - **Foreign Key approach:** The primary key of T is added as a foreign key in S. Attributes of R are moved to S (Include any simple attributes of the 1:N relation type as attributes of S.) This is the foreign key approach.
 - The merged relation approach is not possible for 1:N relationships. (Why?)
 - The cross-reference approach might be used if the join cost is worth avoid null storage.
- **Example:** 1:N relationship types **WORKS_FOR**, **CONTROLS**, and **SUPERVISION** in the figure.
 - For **WORKS_FOR** we include the primary key **DNUMBER** of the **DEPARTMENT** relation as foreign key in the **EMPLOYEE** relation and call it **DNO** and for **CONTROLS** primary key **DNUMBER** of the **DEPARTMENT** relation as foreign key in the **PROJECT**

Employee	<u>SSN</u>	Bdate	FName	Lname	Minit	Address	Salary	Sex	Dno
-----------------	-------------------	--------------	--------------	--------------	--------------	----------------	---------------	------------	------------

Project	<u>Pno</u>	PName	Location	Dno
----------------	-------------------	--------------	-----------------	------------

ER-to-Relational Mapping Algorithm (contd.)

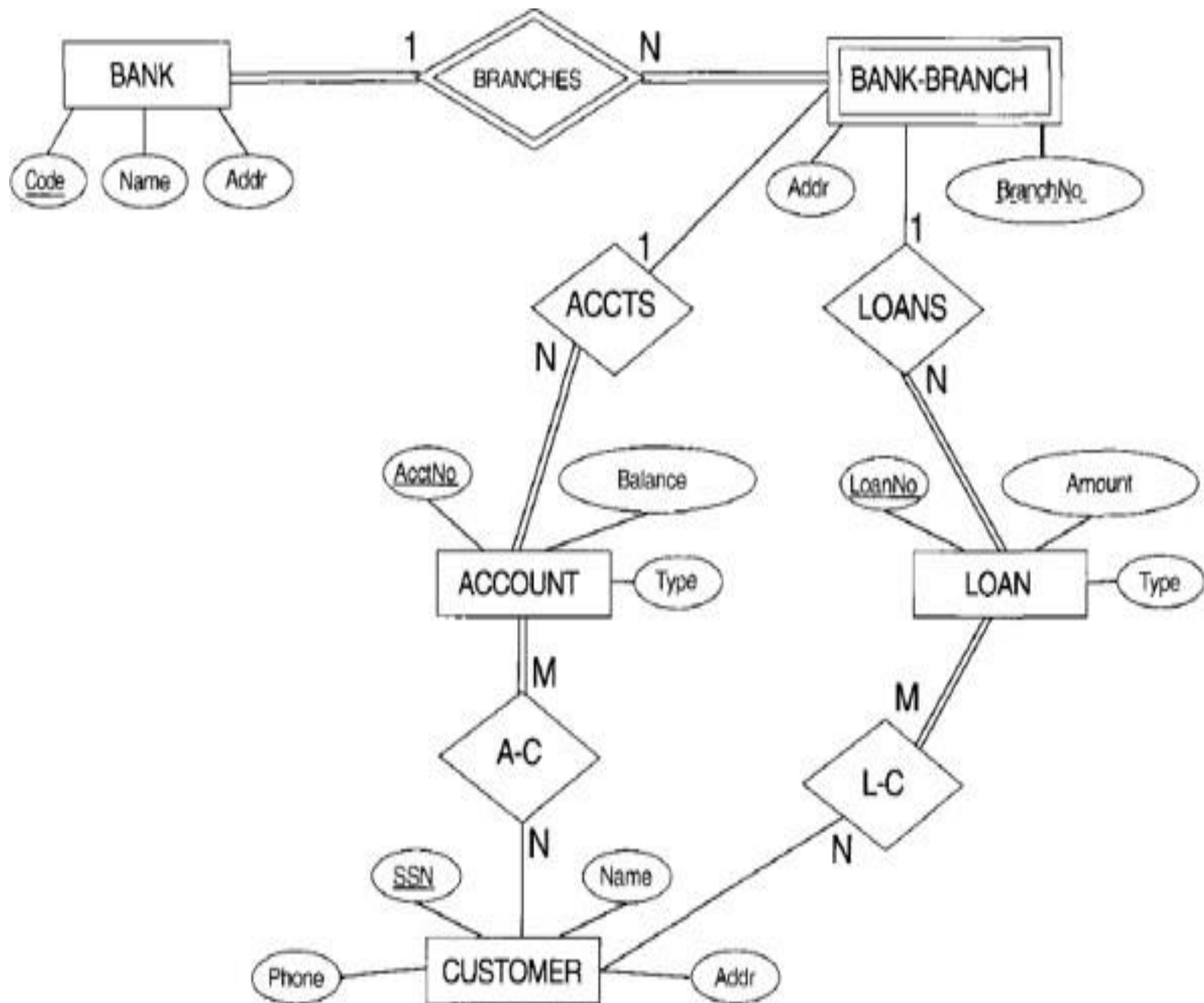
- **Step 5: Mapping of Binary M:N Relationship Types.**
 - The cross-reference approach. For each regular binary M:N relationship type R, ***create a new relation S to represent R.***
 - Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types; *their combination will form the primary key* of S.
 - Also include any simple attributes of the M:N relationship type (or simple components of composite attributes) as attributes of S.
- **Example:** The M:N relationship type **WORKS_ON** from the ER diagram is mapped by creating a relation WORKS_ON in the relational database schema.
 - The **primary keys** of the **PROJECT** and **EMPLOYEE** relations are included as foreign keys in WORKS_ON and renamed **PNO** and **ESSN**, respectively.
 - Attribute **HOURS** in WORKS_ON represents the HOURS attribute of the relation type. The primary key of the WORKS_ON relation is the combination of the foreign key attributes {ESSN, PNO}.

Works-On	<u>ESSN</u>	<u>Pno</u>	Hours
-----------------	--------------------	-------------------	--------------

ER-to-Relational Mapping Algorithm (contd.)

- **Step 6: Mapping of Multivalued attributes.**
 - For each multivalued attribute A, create a new relation R.
 - This relation R will include an attribute corresponding to A, plus the primary key attribute K-as a foreign key in R-of the relation that represents the entity type of relationship type that has A as an attribute.
 - The primary key of R is the combination of A and K. If the multivalued attribute is composite, we include its simple components.
- **Example:** The relation DEPT_LOCATIONS is created.
 - The attribute DLOCATION represents the multivalued attribute LOCATIONS of DEPARTMENT, while DNUMBER-as foreign key-represents the primary key of the DEPARTMENT relation.
 - The primary key of R is the combination of {DNUMBER, DLOCATION}.

DEPT_LOCATIONS	<u>Dno</u>	<u>Dlocation</u>
-----------------------	------------	------------------



ER-to-Relational Mapping Algorithm (contd.)

- **Step 7: Mapping of N-ary Relationship Types.**
 - For each n-ary relationship type R, where $n > 2$, create a new relationship S to represent R.
 - Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
 - Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.
- **Example:** The relationship type SUPPLY in the ER on the next slide.
 - This can be mapped to the relation SUPPLY shown in the relational schema, whose primary key is the combination of the three foreign keys {SNAME, PARTNO, PROJNAME}

EXAMPLE

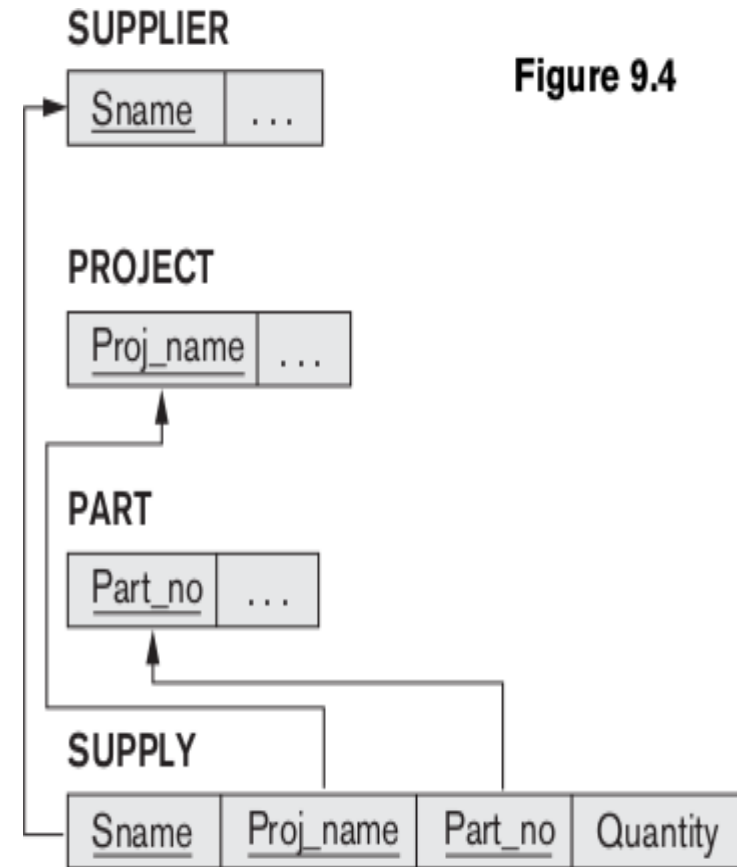
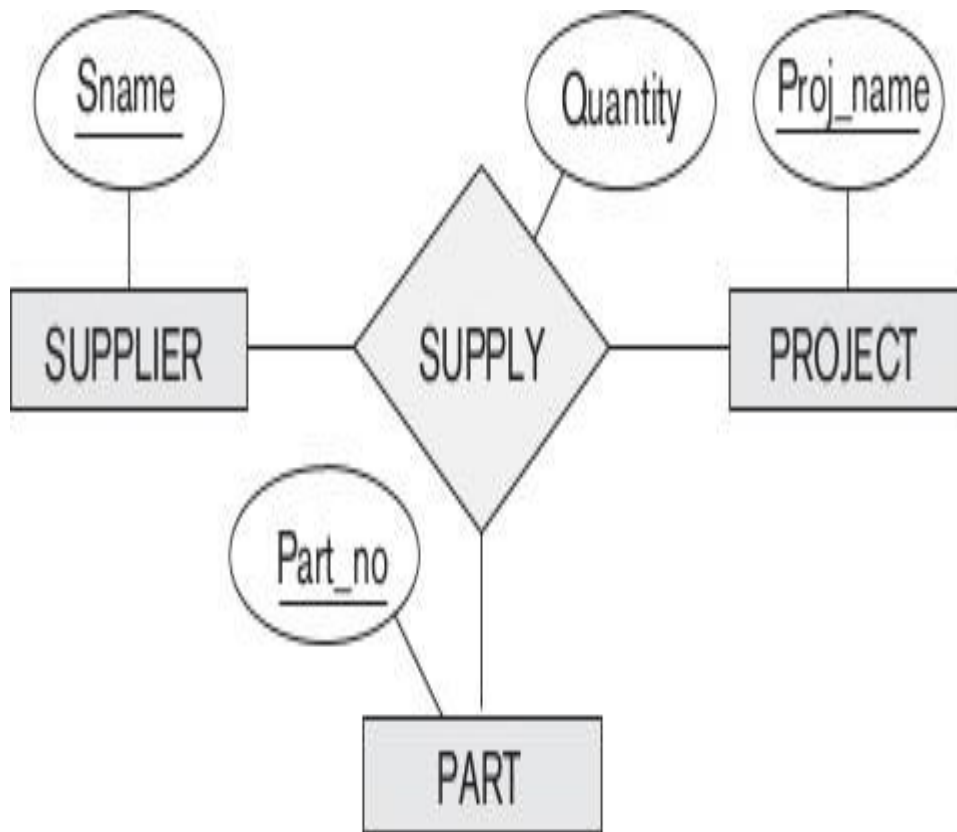


Figure 9.4

Ternary relationship

Summary of Mapping constructs and constraints

Table 7.1 Correspondence between ER and Relational Models

ER Model	Relational Model
Entity type	“Entity” relation
1:1 or 1:N relationship type	Foreign key (or “relationship” relation)
M:N relationship type	“Relationship” relation and two foreign keys
n -ary relationship type	“Relationship” relation and n foreign keys
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation and foreign key
Value set	Domain
Key attribute	Primary (or secondary) key

Mapping EER Model Constructs to Relations

- **Step8: Options for Mapping Specialization or Generalization.**
 - Convert each specialization with m subclasses $\{S_1, S_2, \dots, S_m\}$ and generalized superclass C , where the attributes of C are $\{k, a_1, \dots, a_n\}$ and k is the (primary) key, into relational schemas using one of the four following options:
 - Option 8A: Multiple relations-Superclass and subclasses
 - Option 8B: Multiple relations-Subclass relations only
 - Option 8C: Single relation with one type attribute
 - Option 8D: Single relation with multiple type attributes

Mapping EER Model Constructs to Relations

- **Option 8A: Multiple relations-Superclass and subclasses**
 - Create a relation L for C with attributes $\text{Attrs}(L) = \{k, a_1, \dots, a_n\}$ and $\text{PK}(L) = k$. Create a relation L_i for each subclass S_i , $1 < i < m$, with the attributes $\text{Attrs}(L_i) = \{k\} \cup \{\text{attributes of } S_i\}$ and $\text{PK}(L_i) = k$. This option works for any specialization (total or partial, disjoint or overlapping).
- **Option 8B: Multiple relations-Subclass relations only**
 - Create a relation L_i for each subclass S_i , $1 < i < m$, with the attributes $\text{Attr}(L_i) = \{\text{attributes of } S_i\} \cup \{k, a_1, \dots, a_n\}$ and $\text{PK}(L_i) = k$. This option only works for a specialization whose subclasses are total (every entity in the superclass must belong to (at least) one of the subclasses).

FIGURE 4.4

EER diagram notation for an attribute-defined specialization on JobType.

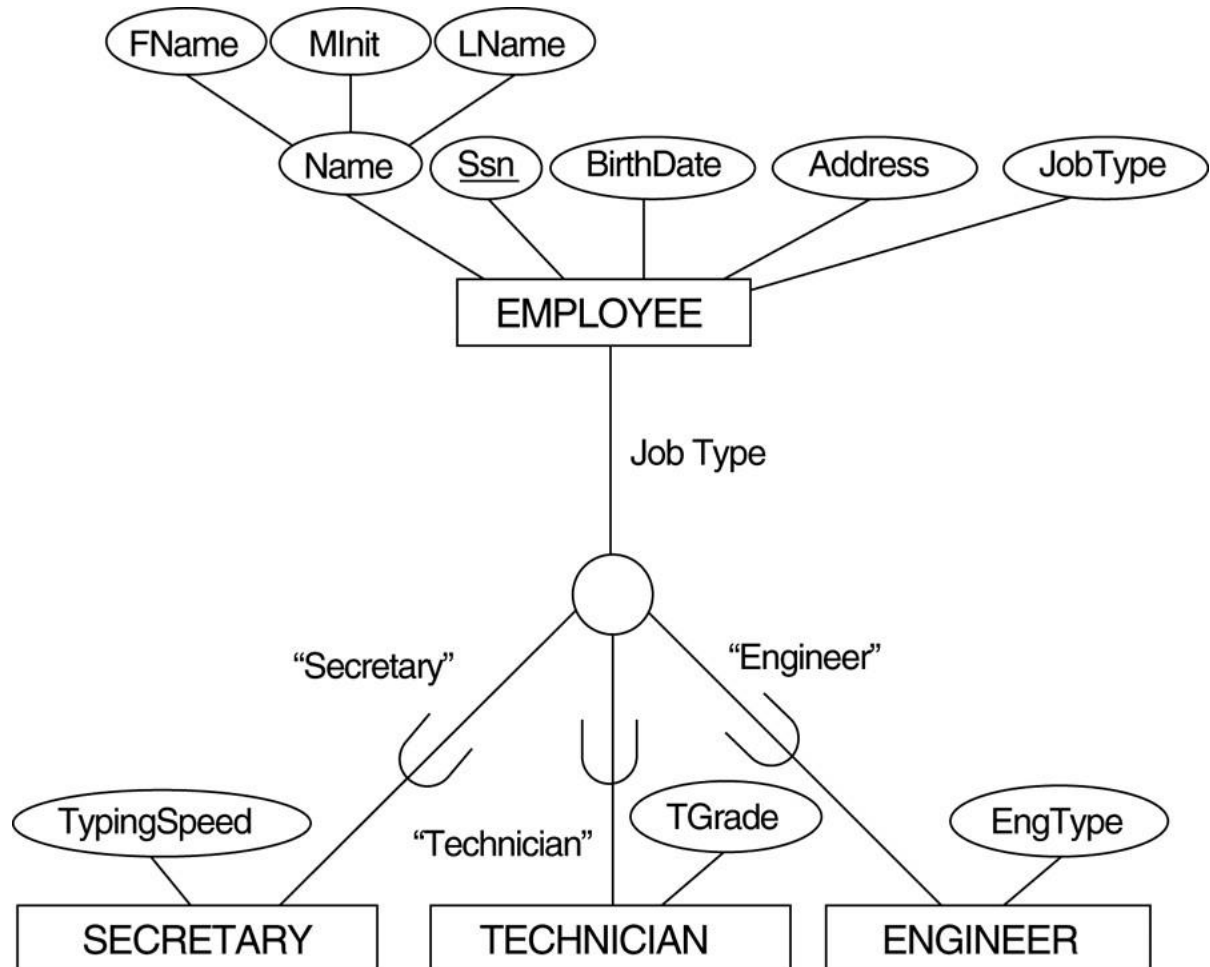


FIGURE 7.4

Options for mapping specialization or generalization.

(a) Mapping the EER schema in Figure 4.4 using option 8A.

(a) EMPLOYEE

<u>SSN</u>	FName	MInit	LName	BirthDate	Address	JobType
------------	-------	-------	-------	-----------	---------	---------

SECRETARY

<u>SSN</u>	TypingSpeed
------------	-------------

TECHNICIAN

<u>SSN</u>	TGrade
------------	--------

ENGINEER

<u>SSN</u>	EngType
------------	---------

FIGURE 4.3

Generalization. (b) Generalizing CAR and TRUCK into the superclass VEHICLE.

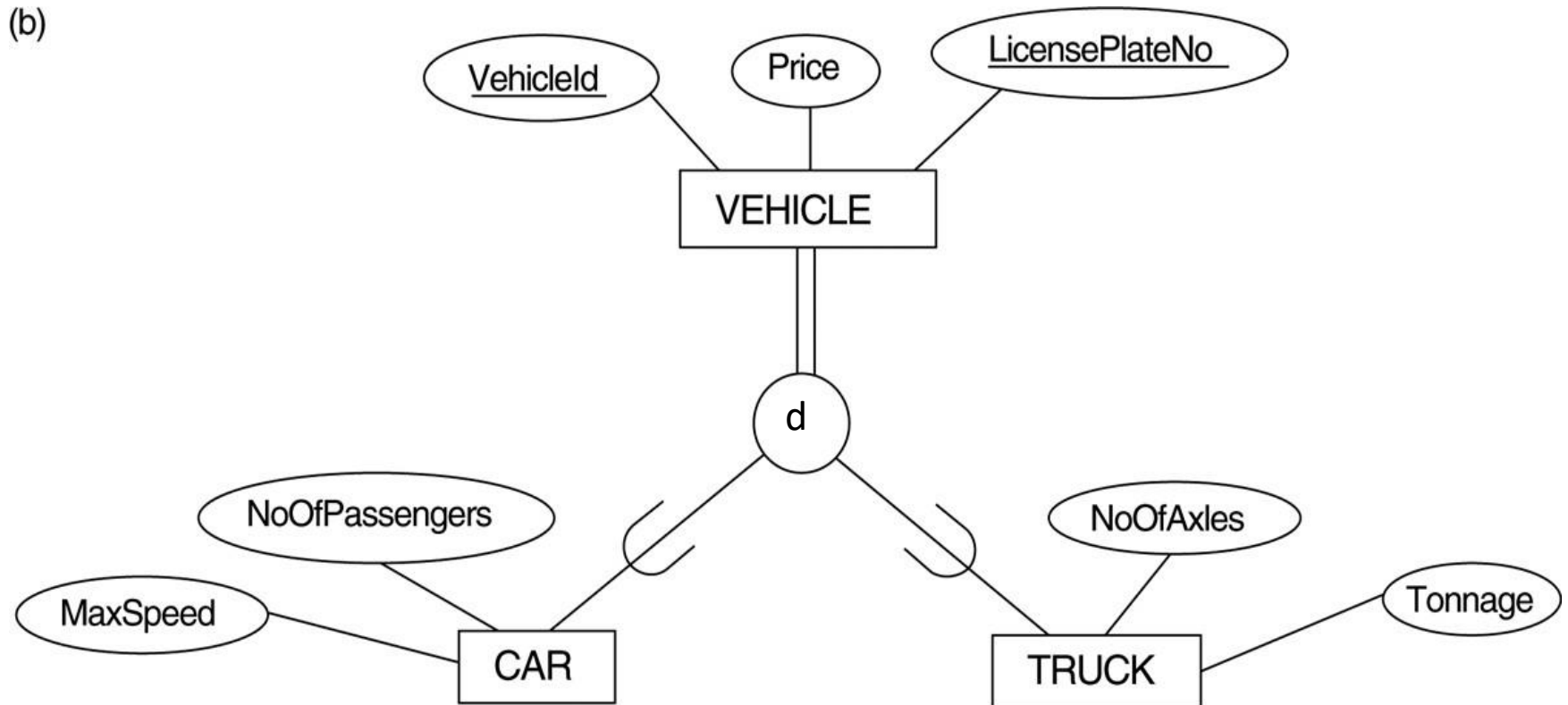


FIGURE 7.4

Options for mapping specialization or generalization.

(b) Mapping the EER schema in Figure 4.3b using option 8B.

(b) CAR

<u>VehicleId</u>	LicensePlateNo	Price	MaxSpeed	NoOfPassengers
------------------	----------------	-------	----------	----------------

TRUCK

<u>VehicleId</u>	LicensePlateNo	Price	NoOfAxles	
------------------	----------------	-------	-----------	--

Mapping EER Model Constructs to Relations (contd.)

- **Option 8C: Single relation with one type attribute**
 - Create a single relation L with attributes $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t\}$ and $\text{PK}(L) = k$. The attribute t is called a type (or **discriminating**) attribute that indicates the subclass to which each tuple belongs
- **Option 8D: Single relation with multiple type attributes**
 - Create a single relation schema L with attributes $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t_1, t_2, \dots, t_m\}$ and $\text{PK}(L) = k$. Each t_i , $1 < i < m$, is a Boolean type attribute indicating whether a tuple belongs to the subclass S_i .

FIGURE 4.4

EER diagram notation for an attribute-defined specialization on JobType.

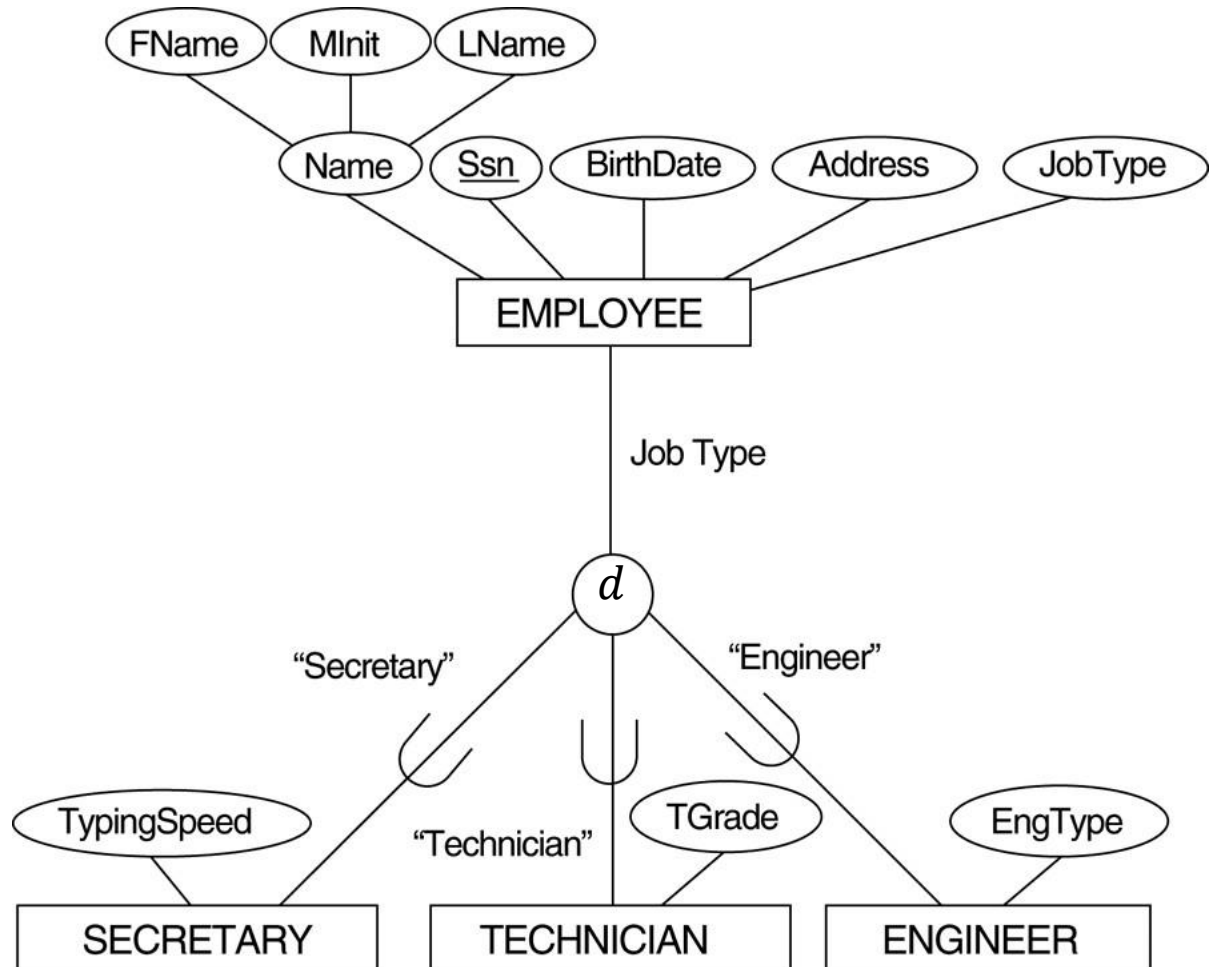


FIGURE 7.4

Options for mapping specialization or generalization.

(c) Mapping the EER schema in Figure 4.4 using option 8C.

(c) EMPLOYEE

<u>SSN</u>	FName	MInit	LName	BirthDate	Address	JobType	TypingSpeed	TGrade	
------------	-------	-------	-------	-----------	---------	---------	-------------	--------	--

FIGURE 4.5

EER diagram notation for an overlapping (non-disjoint) specialization.

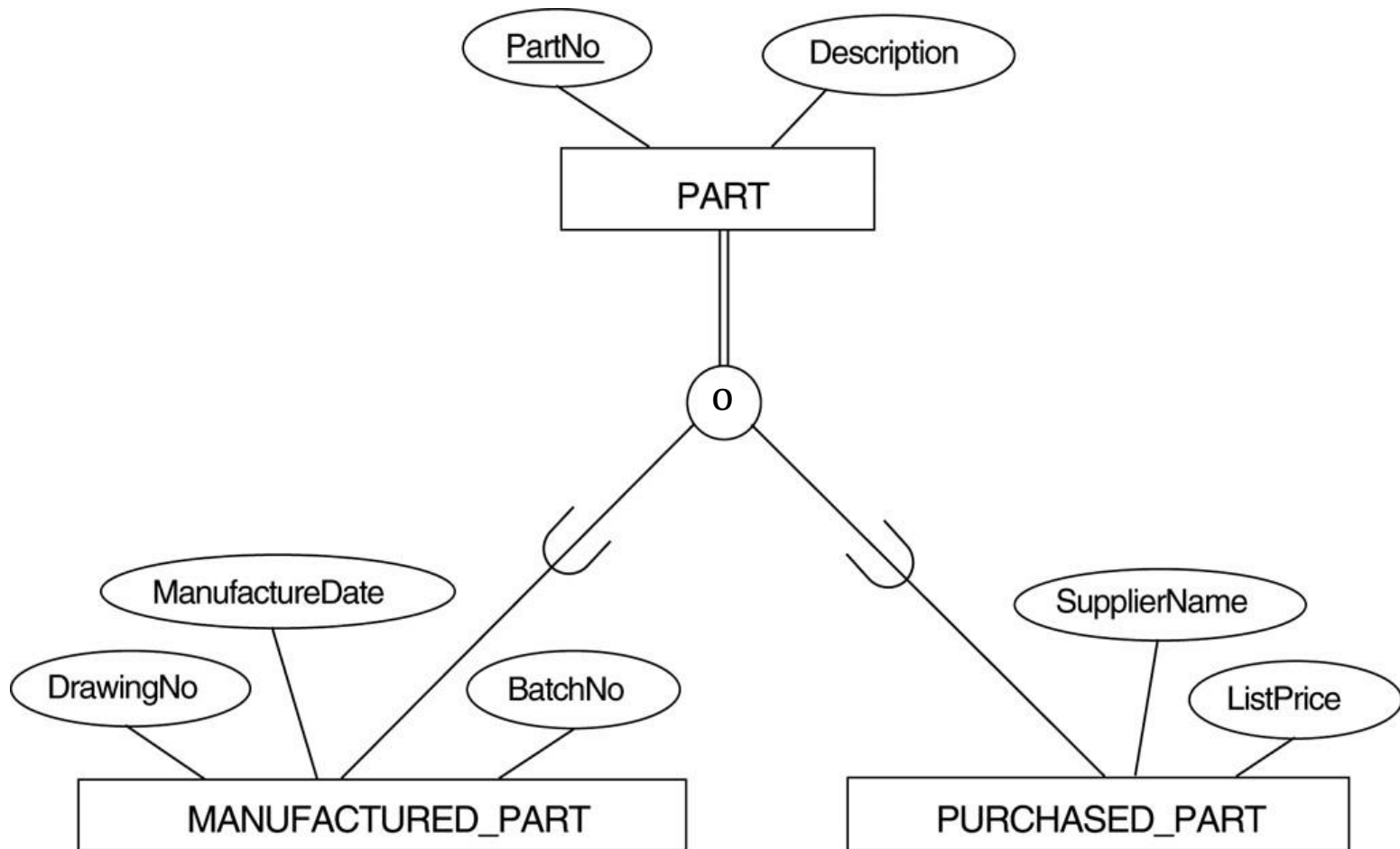


FIGURE 7.4

Options for mapping specialization or generalization. (d) Mapping Figure 4.5 using option 8D with Boolean type fields Mflag and Pflag.

(d) PART

<u>PartNo</u>	Description	MFlag	DrawingNo	ManufactureDate	BatchNo	PFlag	SupplierName	ListPrice
---------------	-------------	-------	-----------	-----------------	---------	-------	--------------	-----------

FIGURE 4.8

Two categories (union types): OWNER and REGISTERED_VEHICLE.

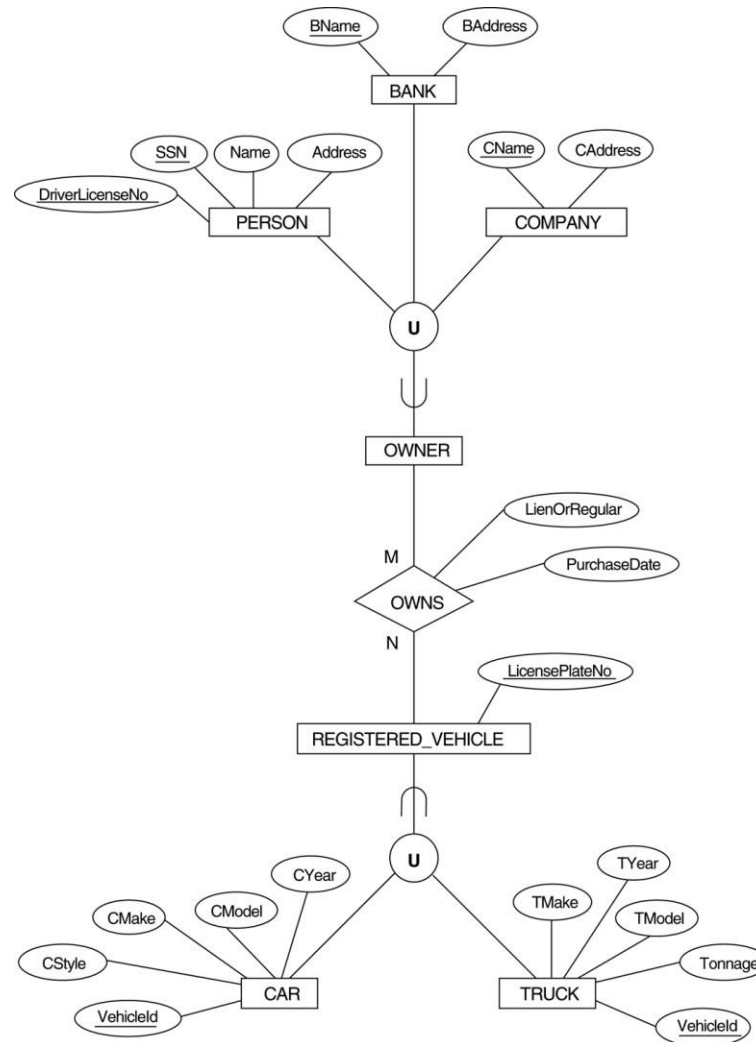


FIGURE 7.6

Mapping the EER categories (union types) in Figure 4.7 to relations.

PERSON

<u>SSN</u>	DriverLicenseNo	Name	Address	
------------	-----------------	------	---------	--

BANK

<u>BName</u>	BAddress	OwnerId
--------------	----------	---------

COMPANY

<u>CName</u>	CAddress	OwnerId
--------------	----------	---------

OWNER

<u>OwnerId</u>

REGISTERED_VEHICLE

<u>VehicleId</u>	LicensePlateNumber
------------------	--------------------

CAR

<u>VehicleId</u>	CStyle	CMake	CModel	
------------------	--------	-------	--------	--

TRUCK

<u>VehicleId</u>	TMake	TModel	Tonnage	TYear
------------------	-------	--------	---------	-------

OWNS

<u>OwnerId</u>	<u>VehicleId</u>	PurchaseDate	LienOrRegular
----------------	------------------	--------------	---------------

Mapping Exercise

Exercise 7.4.

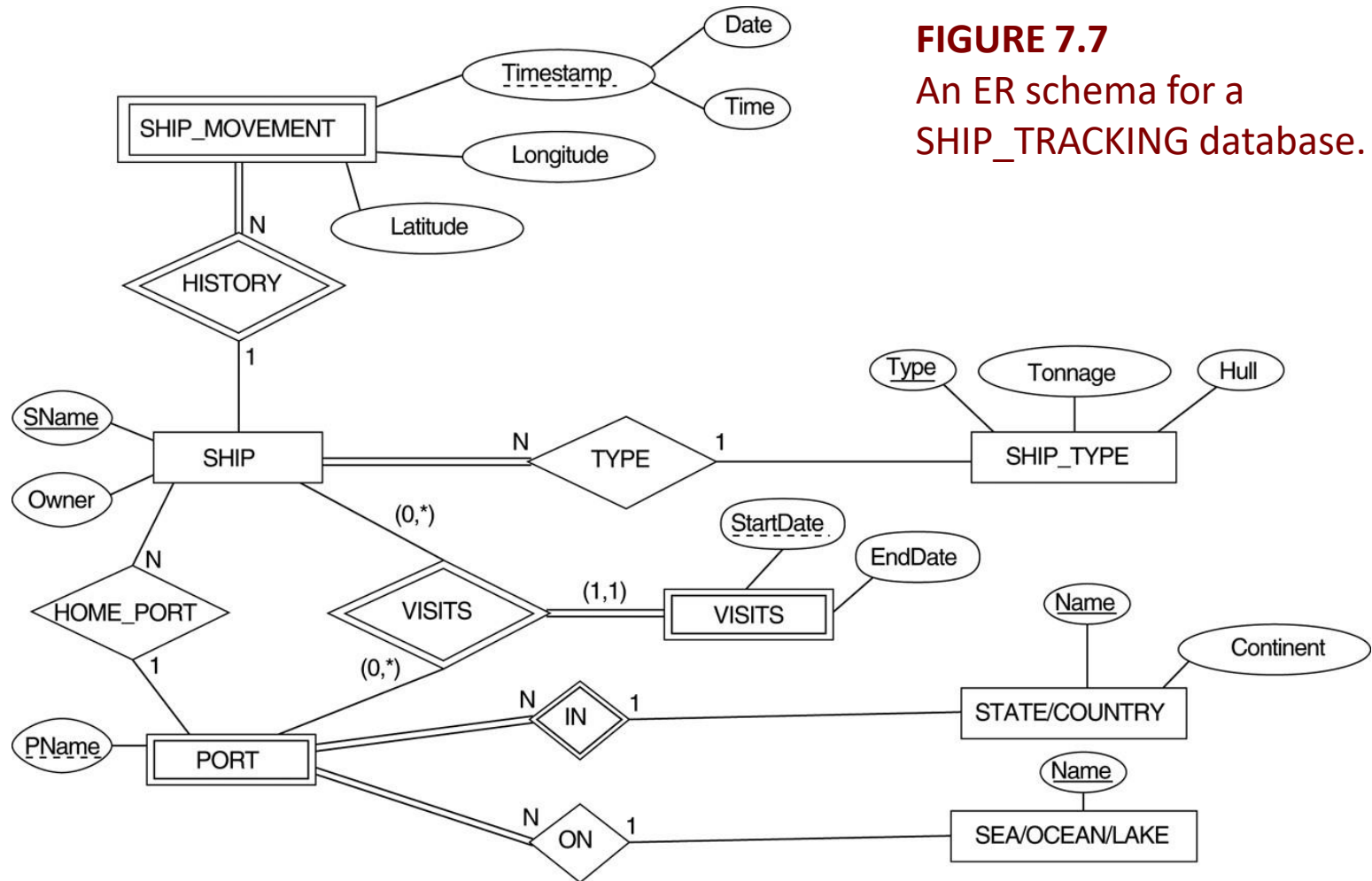


FIGURE 7.7

An ER schema for a SHIP_TRACKING database.

In-Class Exercise

(Taken from Exercise 5.15)

Consider the following relations for a database that keeps track of student enrollment in courses and the books adopted for each course:

STUDENT(SSN, Name, Major, Bdate)

COURSE(Course#, Cname, Dept)

ENROLL(SSN, Course#, Quarter, Grade)

BOOK_ADOPTION(Course#, Quarter, Book_ISBN)

TEXT(Book_ISBN, Book_Title, Publisher, Author)

Draw a relational schema diagram specifying the foreign keys for this schema.

STUDENT

SSN

Name

Major

Bdate

COURSE

Course #

Cname

Dept

ENROLL

SSN

Course #

Quarter

Grade

BOOK_ADOPTION

Course #

Quarter

Book_ISBN

TEXT

Book_ISBN

Book_Title

Publisher

Author

RELATIONAL ALGEBRA OPERATORS

